트레이스 수집 (Trace Collection)

개요

트레이스 수집은 Bus-Mapping 프로세스의 첫 번째 단계로, EVM 실행의 상세한 기록을 수집하는 과정입니다. Scroll ZKEVM은 Geth(Go-Ethereum)의 커스텀 트레이서를 사용하여 블록 실행 중 발생하는 모든 opcode 실행 단계, 상태 변경, 메모리/스택/스토리지 접근을 기록합니다.

이 문서에서는 Geth 트레이서 연동 방식, 트레이스 데이터 구조, 그리고 파싱 과정을 설명합니다.

Geth 트레이서 연동 아키텍처

전체 구조

```
┌──────────────────┐
| Rust 코드 |
| (external- |
| tracer) |
└──────────┬───────┘
| FFI (C ABI)
▼
┌──────────────────┐
| geth-utils |
| (Rust wrapper) |
└──────────┬───────┘
```

```
| CGO
▼
┌──────────────────┐
| Go 코드 |
| (l2geth/ |
| trace.go) |
└──────────┬───────┘
|
▼
┌──────────────────┐
| Scroll Geth |
| (go-ethereum |
| fork) |
└──────────────────┘
```

크레이트 역할

1. **`external-tracer`** : 트레이스 설정을 구성하고 결과를 파싱하는 고수준 Rust API
2. **`geth-utils`** : Go 코드와의 FFI 인터페이스를 제공하는 저수준 래퍼
3. **`l2geth/trace.go`** : Scroll Geth를 사용하여 실제 트레이스를 생성하는 Go 구현

Geth 트레이서 설정

TraceConfig 구조

트레이스 생성을 위한 설정은 `TraceConfig` 구조체로 정의됩니다:

Rust 정의 (`external-tracer/src/lib.rs`):

```rust
#[derive(Debug, Default, Clone, Serialize)]
pub struct TraceConfig {
/// 체인 ID
pub chain_id: u64,
/// 최근 256개 블록 해시 (가장 최근 블록이 마지막)
pub history_hashes: Vec<Word>,
/// 블록 상수 (coinbase, timestamp, number, difficulty, gas_limit, base_fee)
pub block_constants: BlockConstants,
/// 계정 상태 (nonce, balance, code, storage)
pub accounts: BTreeMap<Address, Account>,
/// 실행할 트랜잭션 목록
pub transactions: Vec<Transaction>,
/// 로거 설정 (메모리/스택/스토리지 캡처 여부)
pub logger_config: LoggerConfig,
/// 체인 설정 (하드포크 블록 번호 등)
pub chain_config: Option<ChainConfig>,
/// L1 큐 시작 인덱스 (Scroll 전용)
#[cfg(feature = "scroll")]
pub l1_queue_index: u64,
}
```

Go 정의 (`geth-utils/l2geth/trace.go`):

```
type TraceConfig struct {
ChainID  uint64
StartL1QueueIndex  uint64
HistoryHashes []*hexutil.Big
Block Block
Accounts  map[common.Address]Account
Transactions []Transaction
LoggerConfig *logger.Config
ChainConfig *params.ChainConfig
}
```

LoggerConfig: 트레이스 상세도 제어

`LoggerConfig` 는 트레이스에 포함될 데이터를 제어합니다:

```
#[derive(Debug, Clone, Serialize)]
#[serde(rename_all = "PascalCase")]
pub  struct LoggerConfig {
/// 메모리 캡처 활성화
pub  enable_memory: bool,
/// 스택 캡처 비활성화
pub  disable_stack: bool,
/// 스토리지 캡처 비활성화
```

```
pub disable_storage: bool,
/// 리턴 데이터 캡처 활성화
pub enable_return_data: bool,
}
```

기본값:

- `enable_memory` : feature flag `enable-memory` 에 따라 결정
- `disable_stack` : feature flag `enable-stack` 의 반대값
- `disable_storage` : feature flag `enable-storage` 의 반대값
- `enable_return_data` : 항상 `true`

최적화 팁: 메모리와 스토리지 캡처는 트레이스 크기를 크게 증가시키므로, 필요한 경우에만 활성화해야 합니다.

트레이스 생성 프로세스

1. Rust에서 Go 호출

`external-tracer/src/lib.rs` :

```
pub fn trace(config: &TraceConfig) -> Result<Vec<GethExecTrace>, Error> {
// 1. TraceConfig를 JSON으로 직렬화
let trace_config = &serde_json::to_string_pretty(&config).unwrap();
log::trace!("trace config: {}", trace_config);
// 2. geth-utils를 통해 Go 함수 호출
let trace_string = geth_utils::trace(trace_config)
```

```
.map_err(|error| match error {
geth_utils::Error::TracingError(error) => Error::TracingError(error),
})?;
log::trace!("trace: {}", trace_string);
// 3. JSON 결과를 역직렬화 (재귀 제한 해제)
let mut deserializer = serde_json::Deserializer::from_str(&trace_string);
deserializer.disable_recursion_limit();
let deserializer = serde_stacker::Deserializer::new(&mut deserializer);
serde::Deserialize::deserialize(deserializer).map_err(Error::SerdeError)
}
```

2. FFI 경계 (geth-utils)

geth-utils/src/lib.rs:

```
extern "C" {
fn CreateTrace(str: *const c_char) -> *const c_char;
fn FreeString(str: *const c_char);
#[cfg(feature = "scroll")]
fn CreateL2Trace(str: *const c_char) -> *const c_char;
}

pub fn trace(config: &str) -> Result<String, Error> {
// 1. Rust 문자열을 C 문자열로 변환
```

```
let c_config = CString::new(config).expect("invalid config");
// 2. Go 함수 호출 (unsafe)
let result = unsafe { CreateTrace(c_config.as_ptr()) };
// 3. C 문자열을 Rust 문자열로 변환
let c_result = unsafe { CStr::from_ptr(result) };
let result = c_result
.to_str()
.expect("Error translating EVM trace from library")
.to_string();
// 4. Go가 할당한 메모리 해제
unsafe { FreeString(c_result.as_ptr()) };
// 5. 에러 체크 및 반환
match result.is_empty() || result.starts_with("Failed") {
true => Err(Error::TracingError(result)),
false => Ok(result),
}
}
```

3. Go 트레이스 생성

geth-utils/l2geth/lib.go:

```
//export CreateTrace
func CreateTrace(configStr *C.char) *C.char {
```

```go
// 1. JSON 설정 파싱
var config TraceConfig
err := json.Unmarshal([]byte(C.GoString(configStr)), &config)
if err != nil {
return C.CString(fmt.Sprintf("Failed to unmarshal config, err: %v", err))
}
// 2. 트레이스 실행
executionResults, err := Trace(config)
if err != nil {
return C.CString(fmt.Sprintf("Failed to run Trace, err: %v", err))
}
// 3. 결과를 JSON으로 직렬화
bytes, err := json.MarshalIndent(executionResults, "", " ")
if err != nil {
return C.CString(fmt.Sprintf("Failed to marshal []ExecutionResult, err: %v", er
}
return C.CString(string(bytes))
}
```

4. Scroll Geth 실행

** `geth-utils/l2geth/trace.go` **의 `Trace` 함수는 다음 단계를 수행합니다:

```go
func Trace(config TraceConfig) (*types.BlockTrace, error) {
```

```
// 1. 체인 설정 구성 (하드포크 블록 번호 등)
chainConfig := params.ChainConfig{
ChainID: new(big.Int).SetUint64(config.ChainID),
HomesteadBlock: big.NewInt(0),
// ... 기타 하드포크 설정
ArchimedesBlock: big.NewInt(0), // Scroll 전용
BernoulliBlock: big.NewInt(0), // Scroll 전용
CurieBlock: big.NewInt(0), // Scroll 전용
}
// 2. 트랜잭션 빌드 (L1Msg, PreEip155, Eip155, Eip2930, Eip1559)
txs := buildTxs(config.Transactions, chainConfig.ChainID)
// 3. 블록 컨텍스트 생성
blockCtx := vm.BlockContext{
CanTransfer: core.CanTransfer,
Transfer: core.Transfer,
GetHash: func(n  uint64) common.Hash { /* ... */ },
Coinbase: config.Block.Coinbase,
BlockNumber: toBigInt(config.Block.Number),
Time: toBigInt(config.Block.Timestamp).Uint64(),
// ...
}
// 4. 상태 DB 초기화 (zkTrie 사용)
trieCfg := &trie.Config{IsUsingZktrie: true}
```

```
stateDB, _ := state.New(
common.Hash{},
state.NewDatabaseWithConfig(rawdb.NewMemoryDatabase(), trieCfg),
nil)
// 5. 계정 상태 설정
for address, account := range config.Accounts {
stateDB.SetNonce(address, uint64(account.Nonce))
stateDB.SetCode(address, account.Code)
stateDB.SetBalance(address, toBigInt(account.Balance))
for key, value := range account.Storage {
stateDB.SetState(address, key, value)
}
}
// 6. 초기 상태 커밋
rootBefore, err := stateDB.Commit(block.NumberU64(), true)
// 7. 트레이스 환경 생성
traceEnv := tracing.CreateTraceEnvHelper(
&chainConfig,
config.LoggerConfig,
blockCtx,
config.StartL1QueueIndex,
blockCtx.Coinbase,
stateDB,
```

```
rootBefore,
block,
true,
)
// 8. 블록 트레이스 생성 (실제 EVM 실행)
trace, err := traceEnv.GetBlockTrace(block)
// 9. 최종 상태 커밋
rootAfter, err := stateDB.Commit(block.NumberU64(), true)
// 10. 트레이스에 상태 루트 추가
trace.StorageTrace.RootAfter = rootAfter
trace.Header.Root = rootAfter
return  trace, nil
}
```

GethExecTrace 구조

트레이스 데이터 구조

트레이스 결과는 `GethExecTrace` 구조체로 표현됩니다:

`eth-types/src/lib.rs`:

```
#[derive(Deserialize, Serialize, Clone, Debug, Eq, PartialEq)]
#[serde(deny_unknown_fields)]
pub  struct GethExecTrace {
```

```rust
/// L1 수수료 (Scroll 전용)
#[serde(default)]
pub l1_fee: u64,
/// 사용된 가스
pub gas: Gas,
/// 트랜잭션 실패 여부
pub failed: bool,
/// 실행 결과 (hex 인코딩된 바이트 배열)
#[serde(rename = "returnValue")]
pub return_value: String,
/// 실행 단계 목록 (가장 중요한 필드)
#[serde(rename = "structLogs")]
pub struct_logs: Vec<GethExecStep>,
/// 실행 후 계정 상태 (Scroll 전용)
#[serde(rename = "accountAfter", default)]
pub account_after: Vec<AccountTrace>,
/// 실행 전 상태 (prestate tracer)
pub prestate: HashMap<Address, GethPrestateTrace>,
/// 호출 트레이스 (call tracer)
#[serde(rename = "callTrace")]
pub call_trace: GethCallTrace,
}
```

GethExecStep: 개별 실행 단계

각 opcode 실행은 `GethExecStep` 으로 기록됩니다:

```rust
pub struct GethExecStep {
/// 프로그램 카운터
pub pc: ProgramCounter,
/// 실행된 opcode
pub op: OpcodeId,
/// 남은 가스
pub gas: Gas,
/// 이 단계의 가스 비용
pub gas_cost: GasCost,
/// 환불된 가스
pub refund: Gas,
/// 호출 깊이
pub depth: u16,
/// 에러 (있는 경우)
pub error: Option<GethExecError>,
/// 스택 상태 (feature flag에 따라)
#[cfg(feature = "enable-stack")]
pub stack: Stack,
/// 메모리 상태 (feature flag에 따라)
#[cfg(feature = "enable-memory")]
```

```
pub memory: Memory,
/// 스토리지 상태 (feature flag에 따라)
#[cfg(feature = "enable-storage")]
pub storage: Storage,
}
```

GethExecError: 실행 에러

트레이스는 다양한 실행 에러를 기록할 수 있습니다:

```
pub enum GethExecError {
OutOfGas, // 가스 부족
CodeStoreOutOfGas, // 코드 저장 가스 부족
Depth, // 최대 호출 깊이 초과
InsufficientBalance, // 잔액 부족
ContractAddressCollision, // 컨트랙트 주소 충돌
ExecutionReverted, // 실행 되돌림
MaxInitCodeSizeExceeded, // 최대 초기화 코드 크기 초과
MaxCodeSizeExceeded, // 최대 코드 크기 초과
InvalidJump, // 잘못된 점프
WriteProtection, // 쓰기 보호
ReturnDataOutOfBounds, // 리턴 데이터 범위 초과
GasUintOverflow, // 가스 uint64 오버플로우
InvalidCode, // 잘못된 코드 (0xef로 시작)
```

```
NonceUintOverflow, // nonce uint64 오버플로우
StackUnderflow { stack_len: u64, required: u64 },
StackOverflow { stack_len: u64, limit: u64 },
InvalidOpcode(OpcodeId),
}
```

트레이스 데이터 파싱

JSON 역직렬화

트레이스 데이터는 JSON 형식으로 반환되며, 다음과 같은 특수 처리가 필요합니다:

1. **재귀 제한 해제**: 깊은 호출 스택을 처리하기 위해 `serde_stacker` 사용
2. **대용량 데이터 처리**: 메모리 효율적인 스트리밍 파싱
3. **타입 변환**: Go의 `hexutil.Big` 을 Rust의 `U256` 으로 변환

```
// 재귀 제한 해제를 통한 안전한 역직렬화
let mut deserializer = serde_json::Deserializer::from_str(&trace_string);
deserializer.disable_recursion_limit();
let deserializer = serde_stacker::Deserializer::new(&mut deserializer);
serde::Deserialize::deserialize(deserializer).map_err(Error::SerdeError)
```

Scroll 전용: BlockTrace 변환

Scroll 모드에서는 `BlockTrace` 를 `Vec<GethExecTrace>` 로 변환합니다:

```rust
#[cfg(feature = "scroll")]
pub fn trace(config: &TraceConfig) -> Result<Vec<GethExecTrace>, Error> {
// 1. L2 트레이스 생성
let block_trace = l2trace(config)?;
// 2. ExecutionResult를 GethExecTrace로 변환
Ok(block_trace
.execution_results
.into_iter()
.map(From::from)
.collect::<Vec<_>>())
}
```

ExecutionResult 에서 GethExecTrace 로 변환 (eth-types/src/l2_types.rs):

```rust
impl From<ExecutionResult> for GethExecTrace {
fn from(e: ExecutionResult) -> Self {
let struct_logs = e.exec_steps
.into_iter()
.map(GethExecStep::from)
.collect();
GethExecTrace {
l1_fee: e.l1_fee.as_u64(),
gas: Gas(e.gas),
```

```
failed: e.failed,

return_value: e.return_value,

struct_logs,

account_after: e.account_after,

// ...

}

}

}
```

트레이스 사용 예시

기본 트레이스 생성

```rust
use external_tracer::{TraceConfig, LoggerConfig};

use eth_types::{Address, Word};

use std::collections::BTreeMap;


// 1. 트레이스 설정 구성

let  config = TraceConfig {

chain_id: 534352, // Scroll Mainnet

history_hashes: vec![],

block_constants: BlockConstants {

coinbase: Address::zero(),
```

```
timestamp: Word::from(1234567890u64),
number: Word::from(1000u64),
difficulty: Word::zero(),
gas_limit: Word::from(30_000_000u64),
base_fee: Word::from(1_000_000_000u64), // 1 Gwei
},
accounts: BTreeMap::new(),
transactions: vec![/* ... */],
logger_config: LoggerConfig::default(),
chain_config: None,
l1_queue_index: 0,
};

// 2. 트레이스 생성
let  traces = external_tracer::trace(&config)?;

// 3. 각 트랜잭션의 트레이스 처리
for (tx_index, trace) in  traces.iter().enumerate() {
println!("Transaction {}: {} steps", tx_index, trace.struct_logs.len());
println!(" Gas used: {}", trace.gas.0);
println!(" Failed: {}", trace.failed);
// 4. 각 실행 단계 분석
for (step_index, step) in  trace.struct_logs.iter().enumerate() {
```

```
println!(" Step {}: {:?} at PC {}",
step_index, step.op, step.pc.0);
}
}
```

메모리 캡처 활성화

```
let  config = TraceConfig {
// ...
logger_config: LoggerConfig {
enable_memory: true,
disable_stack: false,
disable_storage: false,
enable_return_data: true,
},
// ...
};
```

성능 고려사항

트레이스 크기

트레이스 크기는 다음 요인에 따라 크게 달라집니다:

1. **트랜잭션 복잡도**: 더 많은 opcode 실행 = 더 큰 트레이스

2. **메모리 캡처**: 각 단계마다 전체 메모리 스냅샷 저장

3. **스토리지 캡처**: 각 단계마다 변경된 스토리지 슬롯 저장

4. **호출 깊이**: 깊은 호출 스택 = 더 많은 단계

최적화 전략:

- 필요한 경우에만 메모리/스토리지 캡처 활성화
- 배치 처리로 여러 트랜잭션을 한 번에 트레이스
- 트레이스 결과를 스트리밍 방식으로 처리

메모리 사용

대용량 트레이스는 메모리 부족을 유발할 수 있습니다:

```
// 재귀 제한 해제로 스택 오버플로우 방지
deserializer.disable_recursion_limit();
let deserializer = serde_stacker::Deserializer::new(&mut deserializer);
```

빌드 설정

Go 의존성

`geth-utils` 는 Go 1.18 이상이 필요합니다:

```
// build.rs
if let Err(e) = build.files(target_files).try_compile(lib_name) {
if format!("{e}").starts_with("Failed to find tool.") {
fail("Failed to find Go. Please install Go 1.18 or later");
```

```
}

}
```

Feature Flags

트레이스 상세도를 제어하는 feature flags:

- `enable-memory` : 메모리 캡처 활성화
- `enable-stack` : 스택 캡처 활성화
- `enable-storage` : 스토리지 캡처 활성화
- `scroll` : Scroll 전용 기능 활성화 (L1Msg 트랜잭션, L1 수수료 등)

```
[features]

default = []

scroll = []

enable-memory = []

enable-stack = []

enable-storage = []
```

다음 단계

트레이스가 수집되면, Bus-Mapping의 다음 단계로 진행됩니다:

1. **RW 연산 생성**: 트레이스에서 Read-Write 연산 추출
2. **Opcode 처리**: 각 opcode별 상세 처리 로직
3. **CircuitInputBuilder**: 트레이스를 회로 입력으로 변환

참고 자료

- **코드 참조**:
- `external-tracer/src/lib.rs` : 트레이스 API
- `geth-utils/src/lib.rs` : FFI 래퍼
- `geth-utils/l2geth/trace.go` : Go 트레이서 구현
- `eth-types/src/lib.rs` : 트레이스 데이터 구조
- **관련 문서**:
- Bus-Mapping 개요
- 데이터 흐름
- 크레이트 구조